Jaxer Guide

- Introduction
- Setup
- Develop
- Debug

Sandbox

Jaxer's secure sandbox lets you load, on your server, pages from other domains and execute the remote page's JavaScript too without giving it access to the Jaxer API or your own server-side code, even though your code has access to their window objects and anything inside them. While you can't visualize these windows (they're on the server after all) you can use them programmatically whether you're:

- HTTP GET: consuming web pages in mashups with very fine-grained control over the processing, block by block
- HTTP POST: scrape the remote DOM, execute its JavaScript functions on your server and post back a response

Since Ajax is so prevalent today in web pages, being able to trigger the Ajax events server-side such that the content added via the Ajax calls is available in a server-side DOM is very handy and can be applied for tasks like search engine optimization and accessiblity modes for your web apps.

Contents

Core Features
Getting a Sandbox Instance
Sandbox Events
With Ajax Libraries
Sandbox Examples
Load Pages Asynchronously and Stream to Browser as Bits Arrive
Get Data Securely
Get Data Securely (Securely)
Using Sandbox to proxy a webpage through Jaxer

API Links

Jaxer.Sandbox
Jaxer.Sandbox.OpenOptions

The Difference between Jaxer.Sandbox and Jaxer.Web.get

`Jaxer.Web.get` will use XMLHTTPResponse and fetch the content returning it as a string or XML object, `Jaxer.Sandbox` is more like 'opening a new tab' inside Jaxer that the page is loaded into. While this page is isolated from the the rest of the instance you can access its DOM and scrape or modify content from it using other Jaxer functionality, plain JavaScript or an Ajax library such as jQuery or Dojo. The result of the processing can be available through a proxied function to the client side.

Core Features

- getElementsByClassName is natively implemented (see John Resig's speed comparison)
- XPath
- Complete Gecko DOM

Getting a Sandbox Instance

The constructor takes three pieces of information: the URL to load, a query string of data to POST and an options hash specifying how to load this Sandbox; all three are optional and leaving the query string empty/null signals Jaxer to do a GET rather than POST.
Tip: Use Jaxer.Util.URL.hashToQuery to build the query string parameter.
You can use `Jaxer.Sandbox.OpenOptions` to control aspects of the Sandbox's behavior, such as whether to execute JavaScript, follow meta redirects and allow the Sandboxed page to itself load frames or iframes.
`Jaxer.Sandbox.readyState` and `Jaxer.Sandbox.waitForCompletion()` to allow you to monitor and work with the loading state.

readyState values:

- 0 UNINITIALIZED—the contents have not been set or the url has not been loaded
- 1 LOADING—the contents are being set or the url is being loaded
- 2 LOADED—the contents have been set or the url has been loaded
- 3 INTERACTIVE—all network operations have completed
- 4 COMPLETED—all operations have completed

`Jaxer.Sandbox.status` and `Jaxer.Sandbox.statusText` give you the HTTP status and HTTP status text, respectively, of the response to the request to fetch this Sandbox's URL, such as 200/OK, 404/Resource Not Found or 500/Server Error. `Jaxer.Sandbox.method` returns the HTTP method (action) of the request: GET, PUT, POST, DELETE.
The `Jaxer.Sandbox.requestHeaders` and `Jaxer.Sandbox.responseHeaders` properties let you examine what was sent and what came back

Sandbox Events

Catch SSL certificate failures inside the Sandbox

Jaxer.Sandbox.OpenOptions allows you to specify a callback function to handle SSL certificate failures in the same way as on server-side XHR requests (Handling SSL Errors). Your callback function is called with the following arguments:

- certInfo: a Jaxer.Util.Certificate.CertInfo object describing an SSL certificate, the connection which returned it and its status
- cert: a Jaxer.Util.Certificate object describing the certificate and its status

- socketInfo
- sslStatus
- targetSite: the https URL requested

The function should return `true` to ignore error and continue page loading, or `false` to abort request. The example below can be combined with the DOM Scraping Example.

```
<script runat="server">
var sandbox = new Jaxer.Sandbox();
var options = {
    onsslcerterror: function(certInfo, cert, socketInfo, sslStatus, targetSite) {
        return !certInfo.isDomainMismatch && certInfo.isSelfSigned;
    }
};
sandbox.open("https://site_with_self_signed_cert.com/index.html", null, options);
</script>
```

Catch location changes inside the Sandbox

If the Sandbox's URL is going to changed due to a server HTTP or HTML <meta> tag redirect or from JavaScript code (if allowed by the `allowJavaScript` option value), you can trap the redirect and evaluate by specifying a callback. The function can return true to allow the redirect and false to block it.

```
<script runat="server">
var sandbox = new Jaxer.Sandbox();
var options = {
    onlocationchange: function(newURL, isHTTPRedirect) {
        return isHTTPRedirect || newURL.substr(0, 22) == "http://www.aptana.com/";
    }
};
sandbox.open("http://www.aptana.com/page_with_redirect.html", null, options);
</script>
```

With Ajax Libraries

This functionality can work well with Ajax libraries too. For instance, jQuery can take a document object as a parameter when generating a selector, so all the related capabilities and plugins can run against a Sandboxed document as well:

```
var sandbox = new Jaxer.Sandbox();
sandbox.open('http://URL');
var title = $("title", sandbox.document).text();
```

Sandbox Examples

Load Pages Asynchronously and Stream to Browser as Bits Arrive

Taken from DOM Scraping with Jaxer part 2

```
<html>
    <head>
        <title>DOM Scraping 1.0</title>
    </head>
    <body>
        <script>
            // On callback, append the results to this page
            function appendHTML(html)
            {
                document.body.innerHTML += html;
            }
            // onload is used to make sure getFragment is defined before it's called:
            window.onload = function()
            {
                Jaxer.Callback.TIMEOUT = 30 * 1000;
                getFragment.async(appendHTML, "Ajaxian Activity Stream", "http://ajaxian.com",
                ↵ true, "activitystream");
                getFragment.async(appendHTML, "News.com Popular Stories",
                ↵ "http://www.news.com/", false, "mostPopStories", "datestamp");
                getFragment.async(appendHTML, "Washington Post",
                ↵ "http://www.washingtonpost.com/", true, "top-box-in");
            }
        </script>
        <script runat="server">
            // Gets a fragment of the remote page's HTML, after some cleanup
            function getFragment(title, url, isClassName, identifier, classesToRemove)
            {
                var sandbox = new Jaxer.Sandbox(url);
                var contents = sandbox.document[isClassName ? 'getElementsByClassName' :
                ↵ 'getElementById'](identifier);
                var container = addToPage(title, contents);
                if (classesToRemove)
                {
                    if (typeof classesToRemove == "string") classesToRemove =
                    ↵ [classesToRemove];
                    classesToRemove.forEach(function(className)
                    {
                        removeNodeList(container.getElementsByClassName(className));
                    });
                }
                return container.innerHTML;
            }
            getFragment.proxy = true;

            function addToPage(title, contents)
            {
                var div = document.createElement('div');

                var h2 = document.createElement('h2');
                h2.innerHTML = title;
```

```
                div.appendChild(h2);

                if (!(contents instanceof HTMLElement)) contents = contents[0];
                div.appendChild(document.importNode(contents, true));

                document.body.appendChild(div);

                return div;
            }

            function removeNodeList(nodeList)
            {
                while (nodeList.length > 0) nodeList[0].parentNode.removeChild(nodeList[0]);
            }
        </script>
    </body>
</html>
```

Get Data Securely

Loads a remote URL in a sandbox, finds all of the h2 elements, and adds them to the body of the requested page.

```
<html>
    <head>
        <script type="text/javascript" runat="server">

            window.onserverload = function() {

                var sandbox = new Jaxer.Sandbox();
                sandbox.open("http://www.aptana.com/jaxer/faq/");
                var body = document.getElementsByTagName('BODY')[0];

                var headers = sandbox.document.getElementsByTagName('H2');
                for (var i = 0; i < headers.length; i++) {
                    var header = headers[i];
                    body.appendChild(header);
                }

                sandbox.close();
            }
        </script>
    </head>
    <body>

    </body>
</html>
```

Get Data Securely (Customized)

```
<html>
    <head>
        <script type="text/javascript" runat="server">

            window.onserverload = function() {

                var sandbox = new Jaxer.Sandbox();
                var openOptions = new Jaxer.Sandbox.OpenOptions();
                openOptions.async = true;
                openOptions.onload = function() {

                    var body = document.getElementsByTagName('BODY')[0];
                    var headers = sandbox.document.getElementsByTagName('H2');
                    for (var i = 0; i < headers.length; i++) {
                        var header = headers[i];
                        body.appendChild(header);
                    }
                    sandbox.close();
                }
                sandbox.open("http://www.aptana.com/jaxer/faq/", null, openOptions);
            }

        </script>
    </head>
</html>
```

Using Sandbox to proxy a webpage through Jaxer

This code loads a remote webpage (http://www.aptana.com) and serves it through the local jaxer instance. It uses jQuery to rebase any relative urls to fully qualified paths. It demonstrates manipulating the DOM on the sandbox to process a page prior to re-sending through Jaxer.

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
    "http://www.w3.org/TR/html4/loose.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
    <head>
        <meta http-equiv="Content-Type" content="text/html; charset=utf-8"/>
        <title>Proxy webpage via Sandbox</title>
        <script src="http://jaxer.org/guide/path/to/jquery.js" runat="server"></script>
    </head>
    <body>
        <h1>Proxy webpage via Sandbox</h1>
        <script runat="server">

          var url = 'http://www.aptana.com/'
          var sb = new Jaxer.Sandbox(url);

          // rebase all relative urls to the original page
          $.each(['href','src'],
             function(index, attribute) {
                $('['+attribute+']',sb.document).each(
                   function(i){
```

```
21.                     var attr = $(this).attr(attribute);
22.                     if ( attr.indexOf('http') == -1 )
23.                     {
24.                        Jaxer.Log.info(attribute+" > "+url+attr);
25.                        $(this).attr(attribute, url+attr);
26.                     }
27.                  })
28.            });
29.
30.          Jaxer.response.setContents(sb.toHTML());
31.        </script>
32.    </body>
33. </html>
```